# Software Testing (CSA37)

**ASSIGNMENT 1**

***CO1: Testing as an engineering activity, role of process in software quality, testing as a process, basic definitions, software testing principles, tester's role, origins of defects, defect classes, defect repository, defect examples.***

---

**Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)**

**Duration:1week**

---

**Scenario — Hospital Outpatient Appointment Application (MediQueue)**

**A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.**

**The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.**

**Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.**

**Incidents reported after release:**

| ID | Incident |
|---|---|
| I1 | Two patients receive the same token number for the same doctor when they book at the same second. |
| I2 | Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements. |
| I3 | Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood. |
| I4 | Application crashes when a patient name has more than 40 characters. |
| I5 | Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment. |
| I6 | Prescription PDF shows the previous patient's details when two doctors save at the same time. |
| I7 | The user manual tells patients to cancel through "My Visits", but this menu does not exist. |
| I8 | A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value. |
| I9 | SMS reminders are not sent when the hospital's SMS provider changes its API key. |
| I10 | The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network. |

**Defect data for the last release (100 defects):**

| Phase | Defects introduced | Defects detected | Cost to fix per defect (₹) | Remarks |
|---|---|---|---|---|
| Requirements | 30 | 2 | 500 | No formal review |
| Design | 22 | 3 | 1,500 | No formal review |
| Coding | 38 | 15 | 3,000 | Developer self-check only |
| Testing | 5 | 20 | 8,000 | Demo-based testing |
| Documentation | 5 | 0 | – | – |
| Production (after release) | – | 60 | 40,000 | Reported by patients |

**Design Challenge:**

**The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.**

**Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.**

---

## TASK 1 – Evaluate Testing as an Engineering Activity 5 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**Investigate whether the way MediQueue was tested can be called an engineering activity.**

**Did the MediQueue team engineer quality into the product, or did they only hope for it?**

**Discuss:**

- **The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)**
- **Which of these characteristics were missing in the MediQueue project, with evidence from the case**
- **How the absence of each characteristic contributed to the incidents reported**
- **What an engineered testing approach would have looked like for this project**

**You are expected to judge the project against engineering practice, not merely define the term.**

**Evidence Expected**

**A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.**

---

## TASK 2 – Evaluate the Role of Process in Software Quality 2 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**The hospital management believes that hiring "better developers" will solve the quality problem.**

**Is quality mainly a result of good people, or of a good process?**

**Discuss:**

- **The role of a defined process in achieving software quality**

- **Whether better developers alone would have prevented the incidents**
- **How a process maturity view (for example, TMM levels) would place the current team**

**Your position must be defended with evidence from the case.**

**Evidence Expected**

**A justified position on the role of process in quality, supported by the case data.**

---

## TASK 3 – Evaluate Testing as a Process 6 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**In MediQueue, testing was a single activity done just before release.**

**What would change in the defect data if testing had been treated as a process running through the whole life cycle?**

**Discuss:**

- **Testing as a process with planning, design, execution, evaluation and reporting activities**
- **The relationship between verification and validation activities and each development phase**
- **The phase in which each type of MediQueue defect should have been detected**
- **Using the defect data table, the cost the hospital paid because defects were found late**
- **The likely saving if requirements and design reviews had been conducted**
- **The limits of your estimate and the assumptions it depends on**

**You must show your calculations and interpret them. Stating that "early testing is cheaper" will not receive full credit.**

**Evidence Expected**

**A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.**

---

## TASK 4 – Apply and Critique Basic Definitions 4 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**The team uses the words "bug", "error" and "failure" interchangeably in its reports.**

**Does careless terminology actually harm quality, or is it only a matter of words?**

**Discuss:**

- **The meaning of error, fault (defect), failure, test case, test oracle and test bed in the MediQueue context**
- **For incidents I3, I5 and I6, the error, the fault and the failure**
- **Why the confusion in terminology made incident I8 possible**
- **Whether the team needs a common glossary, and what it must contain**

**Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.**

**Evidence Expected**

**Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.**

---

## TASK 5 – Evaluate the Application of Testing Principles 3 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**The developers claim that "we tested everything and found no errors, so the software was correct".**

**Which testing principles prove this claim wrong?**

**Discuss:**

- **At least four software testing principles that apply to the MediQueue situation**
- **For each principle, the incident or decision in the case that violates or confirms it**
- **Which principle, if followed, would have had the largest effect on quality, and why**

**Do not simply list principles. You are expected to relate each principle to evidence.**

**Evidence Expected**

**A justified evaluation of the developers' claim using testing principles linked to evidence from the case.**

---

## TASK 6 – Evaluate the Tester's Role 3 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**MediQueue had no independent tester.**

**Would an independent tester have changed the outcome, or would the same defects still have reached patients?**

**Discuss:**

- **The role and responsibilities of a tester across the life cycle**
- **The advantages and limitations of developers testing their own code**
- **Which incidents a tester would most likely have caught, and at which stage**
- **The qualities a tester for a healthcare application must have**

**Your answer must take a position and defend it.**

**Evidence Expected**

**A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.**

---

## TASK 7 – Trace the Origins of Defects 4 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**Using the incidents reported after release, investigate where each defect originated.**

**Where in the life cycle are MediQueue's defects really coming from?**

**Discuss:**

- **The origin of each incident: requirements, design, code, testing, documentation or environment**
- **The evidence in the case that supports each origin you assign**
- **Incidents that may have more than one origin, and how you decided**
- **What the overall pattern of origins tells the hospital about where to invest effort**

**Where an incident could belong to more than one origin, argue for your choice.**

**Evidence Expected**

**A justified origin for each incident and an interpretation of the overall pattern of defect origins.**

---

## TASK 8 – Evaluate Defect Classes and Severity 4 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**A defect classification scheme helps the team understand the nature of its defects.**

**Which classes of defects are most harmful to patients, and do the team's priorities reflect that?**

**Discuss:**

- **A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)**
- **The class, severity and priority of each incident, with reasons**
- **Why severity and priority may differ for the same defect (for example, I7 and I1)**
- **Which defect class the team should target first, and why**

**Simply placing defects in a table is not sufficient. You are expected to justify each classification.**

**Evidence Expected**

**A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.**

---

### TASK 9 – Design and Appraise a Defect Repository 3 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**The hospital currently records complaints in e-mails and a spreadsheet.**

**What information must a defect repository hold so that the team can learn from its defects and stop repeating them?**

**Discuss:**

- **The fields a defect repository for MediQueue must contain, and why each is needed**
- **How the repository supports defect analysis, process improvement and prevention**
- **The weaknesses of the current e-mail and spreadsheet approach**

**You are expected to justify your repository design, not just list fields.**

**Evidence Expected**

**A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.**

---

## TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**Select three incidents that represent different defect classes.**

**For each selected defect, what single change to the process would have prevented it?**

**Discuss:**

- **The root cause of each selected defect**
- **A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)**
- **The cost and effort of each strategy compared with its benefit**
- **Which strategy gives the best value for the hospital**

**Your recommendation must be supported by evidence from the case and the defect data.**

**Evidence Expected**

**A root-cause analysis of three defects with justified, cost-aware prevention strategies.**

---

## TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.**

**Discuss:**

- **The process changes you recommend and their order of priority**

- **The role of testers, reviews and the defect repository in your plan**
- **The expected effect on the number and cost of defects**
- **Trade-offs, risks and limitations of your plan**
- **Whether the plan remains practical for a small team of five developers**

**Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.**

**Evidence Expected**

**A technically justified quality improvement plan supported by the evidence developed in earlier tasks.**

---

## DAY-4 DESIGN CHALLENGE

### TASK 12 – Re-evaluate and Adapt the Quality Process 8 Marks

***CO1 | Bloom's Level: K5 – Evaluate***

**Changed Requirement**

**The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.**

**At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.**

**Patient health records will now be shared between hospitals.**

**Re-evaluate your original solution in response to these changed conditions.**

**You may:**

**RETAIN → MODIFY → or REDESIGN**

**your original solution.**

**A. Impact Analysis – 2 Marks**

***CO1 | Bloom's Level: K5 – Evaluate***

**Discuss:**

- **What has changed from the original situation**
- **Which assumptions in your quality improvement plan are affected**
- **Why frequent releases change the role of testing and the tester**
- **Why sharing health records raises the severity of some defect classes**

**Evidence Expected**

**A clear analysis of how the changed conditions affect the original quality improvement plan.**

**B. Evaluation of the Original Plan – 2 Marks**

***CO1 | Bloom's Level: K5 – Evaluate***

**Evaluate whether your recommended plan remains appropriate.**

**Discuss:**

- **Whether the planned reviews and test activities fit into a two-week cycle**
- **Which new defect origins are introduced by the external health ID system**
- **Whether your defect repository is still sufficient**

**Evidence Expected**

**A technically supported evaluation of the suitability of the original plan under the new conditions.**

**C. Adaptation or Redesign – 2 Marks**

***CO1 | Bloom's Level: K5 – Evaluate***

**Decide whether to:**

**Retain, modify or redesign the quality process.**

**If you modify or redesign it:**

- **Describe the revised process and the tester's revised role**
- **Explain which testing principles now become more important**
- **Explain how defects from the external system will be classified and tracked**

**If you retain the original plan, provide sufficient justification for why no change is necessary.**

**Evidence Expected**

**An appropriate response to the changed requirement with clear quality-engineering reasoning.**

**D. Evidence-Based Defence – 2 Marks**

***CO1 | Bloom's Level: K5 – Evaluate***

**Defend your final decision using evidence.**

**Use:**

- **The defect data and cost figures**
- **Defect origins and classes**
- **Release frequency**
- **Patient-safety and data-privacy risk**
- **Team size and capacity**

**Your final decision must follow logically from your analysis.**

**Evidence Expected**

**A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.**

---

## MARK DISTRIBUTION

| Task | Marks |
|---|---|
| Task 1 – Evaluate Testing as an Engineering Activity | 5 |
| Task 2 – Evaluate the Role of Process in Software Quality | 2 |
| Task 3 – Evaluate Testing as a Process | 6 |
| Task 4 – Apply and Critique Basic Definitions | 4 |
| Task 5 – Evaluate the Application of Testing Principles | 3 |
| Task 6 – Evaluate the Tester's Role | 3 |
| Task 7 – Trace the Origins of Defects | 4 |
| Task 8 – Evaluate Defect Classes and Severity | 4 |
| Task 9 – Design and Appraise a Defect Repository | 3 |
| Task 10 – Evaluate Defect Examples and Prevention Strategies | 4 |
| Task 11 – Recommend and Defend a Quality Improvement Plan | 4 |
| Subtotal | 42 |
| Task 12 – Day-4 Design Challenge | 8 |

| | |
|---|---|
| **TOTAL** | **50** |

# ANSWERS

**Case data used throughout.** The 100 defects of the last release were introduced as Requirements 30, Design 22, Coding 38, Testing 5 and Documentation 5, and were detected as Requirements 2, Design 3, Coding 15, Testing 20 and Production 60. Only 40 of the 100 defects (40%) were found before release. Wherever a rupee figure is shown it is calculated from the defect data table; assumptions are stated explicitly.

## TASK 1 – Evaluate Testing as an Engineering Activity

**Judgement.** MediQueue testing was not an engineering activity. The team did not engineer quality into the product; it hoped for it. Testing was a single event ("developers tested their own code just before release") and the release criterion was a demo that "showed no errors". An engineering activity is planned, follows defined standards and methods, is measured, and can be repeated by someone else with the same result. MediQueue met none of these four tests, and the defect data proves it: 60 of 100 defects were found by patients.

### 1.1 Characteristics of testing as an engineering activity, and how MediQueue measured up

| Characteristic | What engineering practice requires | Evidence from MediQueue | Incidents it contributed to |
|---|---|---|---|
| Planning | Test objectives, scope, risks, resources, schedule, entry and exit criteria decided before coding ends. | No tester, no plan. Testing happened "just before release". The exit criterion was "the demo showed no errors", which is an absence of observed failures in one showing, not a coverage or risk criterion. | I2, I9, I10: nobody planned tests for business rules, external interfaces or performance, so they were never examined. |
| Standards and methods | Documented test design techniques (equivalence classes, boundary values, decision tables), review checklists, coding and documentation standards. | Remarks column of the defect table: "No formal review" (requirements, design) and "Developer self-check only" (coding). No technique was used to choose test inputs. | I4 (40-character boundary), I5 (age 60 boundary), I3 (rule never reviewed with finance). A boundary-value test would have found I4 and I5 in minutes. |
| Measurement and control | Metrics (defects found per phase, removal efficiency, cost of defects) used to decide whether to release and what to improve. | Nobody measured. The data shows that only 6.7% of requirements defects and 6.0% of design defects were found in their own phase, and that production defects carried 91.9% of the total fix cost (Task 3). The management saw none of this before release. | Every incident: with no measure of "how many defects are probably still inside", release was approved on confidence rather than evidence. |

| Characteristic | What engineering practice requires | Evidence from MediQueue | Incidents it contributed to |
|---|---|---|---|
| Repeatability | Written test cases with inputs, expected results and environment, so that anyone can re-run them after every change. | A demo cannot be repeated or re-run for regression. The only recorded test case in the case study (refund) used a wrong expected value and was never reviewed. | I8 (wrong oracle accepted as "Pass"). I1 and I6 (timing-dependent faults that a one-user demo can never trigger). |

### 1.2 How the absence of each characteristic produced the incidents

- **No planning** meant that the questions that reveal missing rules (what days is the OPD closed? what happens when the SMS provider fails? how fast must a screen load?) were never asked. I2, I9 and I10 are plan-level omissions, not coding slips.
- **No standards** meant that a basic, well-known technique was never applied. Boundary values (name length, age 60) and concurrent-use checks are standard test design; their absence produced I4, I5, I1 and I6.
- **No measurement** meant that the poor state of the product was invisible. With 40% pre-release detection, a competent measurement programme would have shown that roughly 60 defects were still in the product on release day.
- **No repeatability** meant that even one wrong test (I8) could pass unchallenged, and that fixes could not be re-verified.

### 1.3 What an engineered testing approach would have looked like

- A test plan written during requirements, risk-ranked so that payment, token allocation and prescription (patient-safety and money) get the deepest testing.
- Requirements and design reviews with checklists, with finance and OPD administration present.
- Test cases designed with equivalence partitioning and boundary value analysis, each with an expected result traced to a requirement ID (the test oracle).
- Unit tests by developers, plus system, concurrency and performance tests by an independent tester in a controlled test environment that mimics the hospital Wi-Fi.
- Written exit criteria (for example: no open severity 1 or 2 defect, all high-risk tests passed) and a weekly report of defects per phase.

**Conclusion.** Judged against planning, standards, measurement and repeatability, MediQueue scored nil on all four. The release approval was a hope, not an engineering decision.

## TASK 2 – Evaluate the Role of Process in Software Quality

**Position.** Quality is mainly the result of a good process, carried out by competent people. Good people are necessary but not sufficient: people make mistakes at a roughly constant rate, and only a process places a detection step after each kind of mistake.

### 2.1 Why better developers alone would not have prevented the incidents

- **Most defects were born before coding.** 52 of the 100 defects (30 requirements plus 22 design) were introduced before a line of code was written. An excellent programmer cannot code a rule that nobody wrote down (I2) or a tax rule that the business explained wrongly (I3).
- **Six of the ten incidents are not skill problems.** I2 (missing requirement), I3 (misunderstood rule), I7 (manual out of step with product), I8 (wrong oracle), I9 (external change) and I10 (no performance target) arise from missing reviews, missing planning or missing checks. Only I1, I4, I5 and I6 are mainly programming matters, and even there a skilled developer reduces the chance of error but does not remove the need for verification.
- **Economics.** Suppose better developers halved the 38 coding defects (19 fewer). At the average cost of ₹26,105 per defect (₹26,10,500 / 100) this would save roughly ₹4.96 lakh. Requirements and design reviews alone are estimated at ₹7.62 lakh saving in Task 3, they address the 52% that better programmers cannot touch, and they cost only a fraction of a senior developer's salary. The coding-defect assumption here is deliberately generous.

### 2.2 Placing the team on a process maturity scale (Testing Maturity Model, TMM)

TMM Level 1 ("Initial") describes an organisation in which testing is not distinguished from debugging, tests are developed ad hoc after coding, the aim of testing is to show that the software works, and there are no trained testers or quality criteria. MediQueue matches this description point by point: the developers "tested" their own code, the objective was a clean demo, and there was no test plan, tester or measurement. The team is therefore at **TMM Level 1**.

The sensible next target is Level 2 (define testing and debugging goals, start test planning, adopt basic techniques such as boundary value analysis), then Level 3 (testing integrated into the life cycle with an independent test group). TMM places formal review programmes and measurement at Level 4, but MediQueue should not wait: lightweight requirement reviews cost ₹500 per defect found and should begin immediately.

**Conclusion.** The case data supports the process view. Hiring better developers treats one phase (coding, 38% of defects) while the process gaps in requirements, design, test design and release control remain.

## TASK 3 – Evaluate Testing as a Process

### 3.1 Testing as a process

Treated as a process, testing consists of five linked activities that run for the whole life cycle:

- **Planning:** objectives, scope, risk ranking, resources, schedule and exit criteria. For MediQueue: payment, token and prescription are high risk.
- **Design:** deriving test cases from requirements and design with defined techniques, writing each expected result (oracle), preparing test data and the test bed.
- **Execution:** running the tests in the planned environment and logging every deviation.
- **Evaluation:** comparing actual with expected results, deciding if a deviation is a software defect or a test defect, analysing coverage and defect trends.

- **Reporting and closure:** test summary, open defects with severity, release recommendation, lessons recorded in the defect repository.

MediQueue had only a rushed form of execution, in the last days, without the other four.

### 3.2 Verification and validation against each phase, and where each incident should have been found

| Phase | Verification (building it right) | Validation (building the right thing) | Incidents that belong here |
|---|---|---|---|
| Requirements | Requirements review / inspection with checklists; check each rule is complete, unambiguous and testable. | Walk-through with OPD administration, finance and doctors; acceptance tests planned. | I2, I3, I10 (no performance target), I5 (age rule wording) |
| Design | Design inspection: concurrency, data isolation, external interface and failure handling, performance budget. | Prototype or storyboard shown to users; integration and system tests planned. | I1, I6, I9 |
| Coding | Code review, coding standard, static analysis, unit tests with boundary values. | Developer checks that each unit behaves as specified. | I4, I5, I6 (code-level cause) |
| Testing | Peer review of test cases and expected results; traceability to requirements. | System, concurrency, performance and acceptance testing in a realistic test bed. | I8, and the last chance for I1, I6, I10 |
| Documentation | Review of the manual against the working build, executing every step of the manual as a test. | Pilot with a few real patients and front-desk staff. | I7 |
| Operation | Monitoring and alerting on external services; regression on each fix. | Patient feedback fed into the repository. | I9 (key change), I10 (field performance) |

### 3.3 Cost of late detection, calculated from the defect data

**(a) What the hospital actually paid** (defects detected × cost per defect):

| Phase of detection | Defects detected | Cost per defect (₹) | Total cost (₹) |
|---|---|---|---|
| Requirements | 2 | 500 | 1,000 |
| Design | 3 | 1,500 | 4,500 |
| Coding | 15 | 3,000 | 45,000 |
| Testing | 20 | 8,000 | 1,60,000 |
| Production | 60 | 40,000 | 24,00,000 |
| **Total** | **100** | **average ₹26,105** | **26,10,500** |

**Interpretation.** Production defects were 60% of all defects but 91.9% of the total cost (24,00,000 / 26,10,500). A defect found by patients cost 80 times a requirements defect (40,000 / 500), 27

times a design defect and 5 times a testing-phase defect. The cost multipliers by phase are 1 : 3 : 6 : 16 : 80.

**(b) How effective each phase was at removing defects** (defects present in the phase are those introduced plus those carried over from earlier phases):

| Phase | Defects present | Detected | Escaped to next phase | Removal efficiency |
|---|---|---|---|---|
| Requirements | 30 | 2 | 28 | 2 / 30 = 6.7% |
| Design | 22 + 28 = 50 | 3 | 47 | 3 / 50 = 6.0% |
| Coding | 38 + 47 = 85 | 15 | 70 | 15 / 85 = 17.6% |
| Testing | 5 + 70 = 75 | 20 | 55 | 20 / 75 = 26.7% |
| Release | 55 + 5 (documentation) = 60 | 0 | 60 reach patients | Overall before release: 40 / 100 = 40% |

**Interpretation.** No phase removed even 30% of the defects present. The two cheapest phases to remove defects, requirements and design, were the least effective (6.7% and 6.0%). That is exactly the effect of "no formal review".

**(c) The price of lateness.** If every defect had been found in the phase where it was introduced, the cost would have been 30 × 500 + 22 × 1,500 + 38 × 3,000 + 5 × 8,000 + 5 × 500 (documentation, assumed ₹500 each because the table gives no figure) = 15,000 + 33,000 + 1,14,000 + 40,000 + 2,500 = **₹2,04,500**. The hospital paid ₹26,10,500, which is 12.8 times as much; the avoidable premium is **₹24,06,000**. This is an upper limit because no real process detects every defect in its own phase. A more modest comparison: if the 60 production defects had been caught in the testing phase instead, the saving would have been 60 × (40,000 – 8,000) = ₹19,20,000.

### 3.4 Likely saving if requirements and design reviews had been conducted

**Assumptions of the model.** (1) A formal requirements review finds 60% of the requirements defects present and a design inspection finds 50% of those present in design (these are assumed rates for checklist-based inspections). (2) Coding and testing phases keep their observed efficiency (17.6% and 26.7%). (3) Introduced defect counts and unit costs stay as in the table. (4) Review effort is assumed at about ₹60,000 per release (about 100 person-hours at ₹600 per hour). Numbers are rounded to whole defects.

| Phase | Present | Detected | Cost per defect (₹) | Cost (₹) |
|---|---|---|---|---|
| Requirements (60%) | 30 | 18 | 500 | 9,000 |
| Design (50%) | 22 + 12 = 34 | 17 | 1,500 | 25,500 |
| Coding (17.6%) | 38 + 17 = 55 | 10 | 3,000 | 30,000 |
| Testing (26.7%) | 5 + 45 = 50 | 13 | 8,000 | 1,04,000 |
| Production | 37 + 5 = 42 | 42 | 40,000 | 16,80,000 |

| Phase | Present | Detected | Cost per defect (₹) | Cost (₹) |
|---|---|---|---|---|
| **Total** | | **100** | | **18,48,500** |

**Saving** = 26,10,500 – 18,48,500 = **₹7,62,000 (29.2%)**; net of the assumed ₹60,000 review effort, about **₹7,02,000**. Defects reaching patients fall from 60 to 42. Sensitivity: with weak reviews (30% and 30%) the saving is ₹4,22,500 (16.2%); with strong reviews (70% and 70%) it is ₹9,67,000 (37.0%). Whichever case is chosen, reviews pay for themselves many times over.

### 3.5 Limits of the estimate and the assumptions behind it

- The review effectiveness rates (60%, 50%) are assumptions. Real rates depend on reviewer skill, preparation time and checklists, and a new team will start lower, as the weak case shows.
- Cost per defect is an average for each phase. A wrong prescription (I6) and a mis-sorted queue do not cost the same; severity-weighted costs would change the figures.
- The table excludes intangible costs: patient harm, loss of trust, staff time spent on complaints, possible legal exposure. The true saving is therefore understated.
- The data come from one release (100 defects, with only 2 and 3 defects in the first two phases). Small counts give wide uncertainty.
- The model assumes defect injection stays constant, whereas reviews also teach developers and reduce injection. It also assumes testing keeps the same efficiency, whereas fewer defects could help or a false sense of security could hurt.
- Documentation defects have no cost in the table (₹500 assumed), and review effort (₹60,000) is an estimate.

## TASK 4 – Apply and Critique Basic Definitions

### 4.1 The six terms applied to MediQueue

| Term | Meaning | In MediQueue |
|---|---|---|
| Error (mistake) | A human action that produces an incorrect result. | The developer misunderstanding the fee and GST rule; the analyst forgetting to record that the OPD is closed on Sundays. |
| Fault (defect, bug) | An incorrect step, statement or specification inside an artefact, the result of an error. | A fee formula in the payment code that adds ₹50 to the displayed ₹500; a missing "closed day" check in the booking code. |
| Failure | Visible deviation from expected behaviour when a fault is executed. | The patient sees ₹500 but is charged ₹550; a Sunday booking is accepted. |
| Test case | Inputs, preconditions, expected results and post-conditions prepared for a test objective. | "Patient born 01-11-1966, appointment 01-11-2026, book with Dr X: expect senior priority flag and a token in the senior queue." |

| Term | Meaning | In MediQueue |
|---|---|---|
| Test oracle | The source that tells us what the correct result is, so pass or fail can be decided. | The approved fee table signed by finance; the refund rule in the requirements. In I8 the oracle was wrong. |
| Test bed | The environment in which tests run: hardware, devices, software, data, tools and stubs. | A staging copy with test patients, an SMS sandbox, a payment-gateway sandbox, Android, iOS and web clients and a network that behaves like the hospital Wi-Fi. MediQueue had none; it only had a demo. |

### 4.2 Error, fault and failure for incidents I3, I5 and I6

| ID | Error (human cause) | Fault (in the artefact) | Failure (what patients saw) |
|---|---|---|---|
| I3 | The team misunderstood how the tax rule applies to the consultation fee, and the booking page and the payment module were built from different understandings. | The payment code adds tax on top of the base fee, while the booking page shows the base fee only (duplicated price logic that disagrees). | ₹500 displayed, ₹550 charged. |
| I5 | The developer reasoned about "above 60" instead of "60 or older", or computed age on the booking date rather than the appointment date, and the rule was not written precisely. | A comparison such as age > 60 (an off-by-one boundary fault) or an age calculation using the wrong date. | A patient who turns 60 on the appointment day does not receive senior-citizen priority. |
| I6 | The developer assumed that only one doctor saves at a time and did not think about concurrent use. | A shared object, buffer or temporary file used by the PDF generator without isolation or locking. | The prescription PDF of one patient contains the details of another: wrong clinical information and a privacy breach. |

Note that one fault can produce many failures (I6 repeats every time two doctors save together), and one error can create several faults (I3 affected the booking page and the payment module).

### 4.3 How loose terminology made I8 possible

The team used "bug", "error" and "failure" as one word. In I8 the tester ran the refund test, compared the output with an expected value and recorded "Pass". Three different things were being blurred: the **failure** (actual behaviour), the **oracle** (what should happen) and a **fault in the test itself** (a wrong expected value). Because "no error found" and "no bug" meant the same, nobody asked the question that terminology forces: "is the software wrong, or is the expected value wrong?". The expected value was not treated as a controlled item with a source, so it was never reviewed. A test defect, as a category, did not exist in their vocabulary, so it could not be reported. The result was a pass that meant only "the software agrees with a mistake", and the refund fault travelled to production.

### 4.4 Does careless terminology harm quality?

**Yes, it does more than confuse words.** (1) Root-cause analysis needs the chain error, fault, failure: prevention acts on errors (training, reviews), detection acts on failures, and repair acts on faults. If they are mixed, the team patches symptoms. (2) Metrics become unreliable: counting failures reports I6 many times, counting faults reports it once, so defect density and removal efficiency are not comparable between releases. (3) Developers, testers and hospital management cannot agree on severity or on when something is "fixed". (4) As I8 shows, an unnamed concept (test oracle, test defect) is not managed.

**A common glossary is therefore needed.** It should contain: error, fault/defect, failure and the status of "bug" as an informal word; test case, test oracle, test bed, test defect; verification and validation; severity and priority with their scales; defect states (new, assigned, fixed, retested, closed, reopened, rejected); defect classes and origins; pass, fail and blocked; release exit criteria and "done"; and the **hospital terms** that cause defects when ambiguous, such as "senior citizen" (age 60 or above on the appointment date), "OPD working day", "token", "same slot" and "refund". A glossary will not fix quality by itself, but without it the other improvements (repository, metrics, reviews) cannot be applied consistently.

## TASK 5 – Evaluate the Application of Testing Principles

**The developers' claim** ("we tested everything and found no errors, so the software was correct") is wrong on three counts: they did not test everything, finding no errors in a demo does not show there are none, and correctness is not the same as fitness for patients. The principles below show why.

| Principle | Evidence from the case | Effect on the claim |
|---|---|---|
| 1. Testing can show the presence of defects, never their absence. | The demo "showed no errors", yet within three months patients exposed ten different incident types and, per the defect data, 60 defects. | Zero failures in a demo is a weak statement about the product. The claim reverses the principle. |
| 2. Exhaustive testing is impossible; tests must be selected by risk. | 6,000 patients a day, free-text names, any date, any age, simultaneous bookings and saves. The team had no way of testing "everything" and no risk ranking to choose what to test. | "Everything" cannot have been tested. I4 (name length) and I1 and I6 (timing) live in huge input and timing spaces. |
| 3. Test early; testing starts with requirements. | 52 of 100 defects were introduced in requirements and design, and only 5 were found there. I2 and I3 were requirements defects. | A test at the end can only find what survives to the end; requirements defects were invisible to a demo of the built system. |
| 4. Absence of errors is not fitness for use (absence-of-errors fallacy). | I2 (Sunday rule never specified), I3 (rule misunderstood) and I7 (manual describes a missing menu): the software could "work as built" and still be wrong for the patient. | Even a flawless match to a wrong specification is a failed product. |

| Principle | Evidence from the case | Effect on the claim |
|---|---|---|
| 5. A test case must define the expected result (the oracle), which needs checking. | I8: the refund test passed against a wrong expected value. | "No errors found" is only as good as the oracle behind it. |
| 6. Tests should be run by someone independent of the author and must include invalid and unexpected inputs. | Developers tested their own code, and the demo used normal inputs. I4 (over-long name) and I2 (invalid Sunday date) are invalid-input cases. | The authors tested what they had in mind; the defects were in what they had not thought of. |
| 7. Testing is context dependent. | A hospital application handles money, queue fairness, privacy and prescriptions. A demo-level approach suitable for a prototype was used. | Risk is higher than in a consumer app, so the test effort should be higher. |

**Which principle would have had the largest effect? Early testing** (principle 3). The defect data show that 52% of defects originated in requirements and design, that a requirements defect costs ₹500 to fix and a production defect ₹40,000, and that the estimate in Task 3 attributes about ₹7.6 lakh saving to requirement and design reviews alone. It would have prevented I2 and I3 and made I1, I5, I9 and I10 visible during design. The independence and oracle principles (5 and 6) are also important, but they act at the test stage where each defect already costs ₹8,000 or more, and they do not help with defects that never existed in the code.

## TASK 6 – Evaluate the Tester's Role

**Position.** An independent tester would have changed the outcome substantially, but not entirely. I judge that about half of the incidents would very probably have been caught and three more probably, while two (I8 and I9) would only be prevented by the process around the tester. A tester added only at the end of the project would help much less than one who joins at requirements.

### 6.1 Role and responsibilities across the life cycle

- **Requirements:** review for ambiguity and testability, ask "what if" questions (closed days, boundary ages), start acceptance test design.
- **Design:** join design inspections, plan integration, system, concurrency and performance tests, define the test bed.
- **Coding:** agree unit-test and coding standards with developers, review unit-test adequacy.
- **Testing:** design and execute tests, record defects in the repository, retest and run regression, review test cases and expected results.
- **Release and after:** give release evidence (test summary, open defects by severity), analyse escaped defects and feed lessons back into the process.

### 6.2 Developers testing their own code

**Advantages:** immediate feedback, detailed knowledge of the code (good for unit tests and white-box coverage), and the cheapest place to fix a defect (₹3,000 in coding against ₹8,000 in testing and ₹40,000 in production).

**Limitations:** the author who misunderstood a rule writes code and tests from the same misunderstanding (I3). Authors tend to confirm that the code works rather than to break it. Schedule pressure and the "demo with no errors" release rule create a conflict of interest. Developers usually lack test design skills and the viewpoint of the patient or front-desk staff. The data confirm it: self-check detected only 15 of the 85 defects present in coding (17.6%).

Developers should therefore write unit tests, but must not be the only testers.

### 6.3 Incidents a tester would most likely have caught, and when

| ID | Likelihood | How and at what stage | Reason |
|---|---|---|---|
| I2 | High | Requirements review: "Which days is the OPD closed?" | Missing rules show up when test conditions are derived from requirements. |
| I3 | High | Requirements review with finance; system test comparing booking page with the payment amount. | Cross-screen consistency check needs no knowledge of the tax rule. |
| I4 | High | Unit and system test with over-long names (boundary value analysis). | Standard input test. |
| I5 | High | System test with ages 59, 60 and 61 on the appointment date. | Standard boundary test. |
| I7 | High | Documentation test: follow the manual step by step on the build. | Direct comparison of manual and product. |
| I1 | Medium | Design inspection (concurrency checklist), then a scripted parallel-booking test. | Needs the tester to think about simultaneous users and have tools. |
| I6 | Medium | Concurrency test with two doctors saving together, in system test. | Same. |
| I10 | Medium | Requirement for a performance target, then a performance test on a network like the hospital Wi-Fi. | Depends on the test bed and a stated target. |
| I8 | Low | Only through peer review of the test case and its expected value. | The tester made the mistake; independence does not guarantee a correct oracle. |
| I9 | Low–medium | Failure-injection test of the SMS interface in design/system test; monitoring after release. | The trigger (key change) occurred after release; a process, monitoring and design matter more. |

### 6.4 Qualities needed in a healthcare tester

- Domain knowledge of OPD workflow, billing, priority queues and prescriptions, so that expected results are right.
- A patient-safety and privacy mindset, because a wrong prescription (I6) harms people and breaches confidentiality.
- Sceptical, curious thinking with attention to detail, and the independence and courage to say "do not release".
- Technical skills: test design techniques, SQL, API testing, concurrency and performance testing, automation.
- Clear communication and diplomacy, and empathy for patients with low digital skill or age-related limits (senior citizens).

**Judgement.** An independent tester is necessary, because the five high-likelihood incidents above are failures that a test designer thinks of first. It is not sufficient: I8 and I9 show that without reviewed oracles, defined processes and monitoring, some defects still reach patients.

## TASK 7 – Trace the Origins of Defects

| ID | Origin (primary) | Evidence and reasoning | Other origins and how decided |
|---|---|---|---|
| I1 | Design | Token allocation has no mechanism that guarantees uniqueness under simultaneous use; this is a structural omission (no atomic allocation or unique constraint), not a one-line slip. | Coding (the code implements the weak design) and Testing (no concurrency test). Primary is design because a correct implementation of that design would still fail. |
| I2 | Requirements | The case says explicitly that the rule "was never written in the requirements". | Testing also missed it, but the defect was born as an omission. |
| I3 | Requirements | "The GST rule was misunderstood": the interpretation of a business rule is a requirements error. | Design (price calculated in two places). Primary is requirements because the underlying misunderstanding would corrupt any design. |
| I4 | Coding | The application crashes: missing input validation or exception handling. A well-behaved program rejects bad input rather than crashing. | Requirements (field length may not have been specified) and Testing. Coding chosen because a crash is a robustness fault regardless of the stated limit. |
| I5 | Coding | The failure occurs exactly at the boundary (turning 60 today): a typical comparison or date error. | Requirements (rule not stated precisely, "as on which date?"). Coding chosen because "senior citizen = 60 and above" is the common rule; the exact failure is at the code boundary. |
| I6 | Coding | Shared state in the PDF-generation path is overwritten by a second user. | Design (no review of shared resources). Seen as the code-level partner of I1. |

| ID | Origin (primary) | Evidence and reasoning | Other origins and how decided |
|---|---|---|---|
| I7 | Documentation | The manual describes a menu "My Visits" that does not exist. | Weak change control: the product or the manual changed without the other being updated. |
| I8 | Testing | A test case with a wrong expected value was marked Pass. | Requirements (refund rule may have been unclear). Testing chosen because the wrong value was in the test artefact. |
| I9 | Environment (external interface) | The trigger is the provider changing its API key, outside the team's control. | Design (no handling or alert for interface failure). Environment is primary as the cause, with design the team's share of the blame. |
| I10 | Design | A screen that takes 25 s on the hospital Wi-Fi points to a heavy design (data volume, queries, payload). | Requirements (no performance target), Testing (no test on realistic network), Environment (Wi-Fi). Design is the one the team controls. |

**Count of primary origins:** Requirements 2 (I2, I3), Design 2 (I1, I10), Coding 3 (I4, I5, I6), Testing 1 (I8), Documentation 1 (I7), Environment 1 (I9).

**Overall pattern.** If contributing origins are included, **9 of the 10 incidents have a requirements or design contribution** (all except I7), and every incident passed through a testing process that failed to catch it. This agrees with the defect table, where 52% of defects were introduced in requirements and design and 38% in coding. The faults are upstream and the escape is downstream.

**Where to invest.** (1) Requirements and design verification first (reviews with checklists), because that is where most defects arise, it is cheapest (₹500 to ₹1,500) and almost nothing is found there today (6.7% and 6.0%). (2) Coding controls second: coding standard, boundary-based unit tests, concurrency review. (3) Independent test design and reviewed oracles third. (4) Operational resilience: monitoring and alerting for external services. (5) Document review as a small, cheap step.

## TASK 8 – Evaluate Defect Classes and Severity

### 8.1 Classification scheme used

- **Requirements/specification defects:** functional description, feature, feature interaction, interface description.
- **Design defects:** algorithm and processing, control logic and sequence, data, module interface, external interface description.
- **Coding defects:** algorithm and processing, control logic and sequence, data and data flow, initialisation, module interface, typographical, external interface.
- **Testing defects:** test harness and test environment defects, test case design and test procedure defects.

**Severity** measures how bad the effect of the failure is on patients and the hospital: S1 Critical (patient safety, privacy breach, financial loss for many users, outage); S2 High (core function

wrong, no good workaround); S3 Medium (limited function or small group affected, workaround exists); S4 Low (cosmetic or guidance). **Priority** is the order in which the defect should be fixed, set by severity together with frequency, cost of the fix, workarounds and deadlines: P1 immediate hotfix, P2 next release, P3 when convenient.

### 8.2 Classification of the ten incidents

| ID | Class (sub-class) | Sev. | Prio. | Justification |
|---|---|---|---|---|
| I1 | Design (control logic and sequence) | S2 | P1 | Breaks queue fairness for the core function; no data loss. Frequent at peak times for popular doctors with 6,000 patients a day, and the fix (an atomic number or unique constraint) is cheap. |
| I2 | Requirements (functional description: missing rule) | S2 | P1 | Sick patients travel to a closed OPD. Fix is small (block Sunday and rebook existing bookings). |
| I3 | Requirements (feature: wrong business rule); also design | S1 | P1 | Every payment is affected; patients are charged more than they were shown. Direct financial and trust damage and possible legal exposure. |
| I4 | Coding (data / input validation) | S2 | P1 | Crash for any long name, a common situation; users cannot book. Easy to fix. |
| I5 | Coding (control logic: boundary condition) | S3 | P2 | Affects a small group (those turning 60 on the day); wrong queue position, no clinical harm. Cheap fix, schedule in the next release. |
| I6 | Coding (data / data flow, shared state) | S1 | P1 | Wrong patient details on a prescription: clinical risk and disclosure of health data. Needs an immediate hotfix and incident handling. |
| I7 | Documentation (inconsistency with product) | S4 | P3 | Function unaffected; wrong guidance. Fix with next release but costs almost nothing. |
| I8 | Testing (test case design: wrong oracle) | S2 | P2 | Does not harm patients directly but it hid a refund defect. The test must be corrected at once and other expected values audited. |
| I9 | Design (external interface / error handling); environment trigger | S2 | P2 | Reminders fail for every patient, silently. The immediate action (renew the key) is operational, so the permanent fix (alerting and key management) is scheduled. |
| I10 | Design (performance); no requirement | S3 | P2 | Slows work in clinics but does not give wrong results; needs measurement and a target before tuning. |

### 8.3 Why severity and priority differ

Severity describes the damage if the failure occurs; priority is a scheduling decision that adds likelihood, effort, workarounds and business timing. **I7** has the lowest severity (S4) yet needs no development effort beyond editing a document, so it is easy to fix with the next release; if the hospital were about to print thousands of leaflets, its priority would rise although its severity would not. **I1** has high severity because it damages a core function, but its priority depends on frequency:

a rare collision with a manual workaround at the desk could wait, but at 6,000 patients a day and a trivial fix it is P1. The reverse also occurs: a spelling mistake in the hospital name is S4 but may be P1 before a launch, and a crash in a seldom-used report may be S2 but P3. A single scale cannot express both ideas.

### 8.4 Which classes harm patients most, and do the team's priorities reflect it?

The classes that most harm patients are **data and concurrency defects in coding and design (I6, I1)**, because they can produce wrong clinical information and privacy breaches, and **requirements defects (I3, I2)**, because they affect every patient and propagate into design, code, tests and manuals. Documentation and cosmetic defects are least harmful. The team's priorities did **not** reflect this. There was no severity or priority scale, complaints arrived in e-mails and a spreadsheet, and release was decided by a clean demo, which tests normal use and so favours visible, simple behaviour and never shows concurrency, boundary or rule defects.

**Class to target first:** requirements and specification defects. They are 30% of all defects introduced, only 6.7% are found in the phase, they are cheapest to fix at that time (₹500 against ₹40,000 in production), and they contaminate everything built after them. The single most dangerous defect (I6) should in the meantime be hotfixed immediately and design inspections should adopt a concurrency checklist.

## TASK 9 – Design and Appraise a Defect Repository

A defect repository is useful only if it records not just that a defect exists, but where it came from, why it was missed, what it cost and how it was prevented from recurring. The design below is grouped by purpose.

### 9.1 Repository design and justification of fields

| Group | Fields | Why they are needed |
|---|---|---|
| Identification and workflow | Defect ID, title, reporter, date and time, owner, status (new, assigned, fixed, retest, verified, closed, reopened, rejected, duplicate), due date. | A unique ID and status give an audit trail and prevent lost or duplicate complaints; ownership makes someone accountable. |
| Reproduction | Release and build, platform (Android, iOS, web), device/browser, environment (production, staging), steps to reproduce, expected result and its source (requirement or rule ID), actual result, screenshots or logs. | The developer must reproduce the failure. Recording the expected result and its source exposes oracle errors such as I8. |
| Classification | Defect class and sub-class, origin phase (injected), detection phase, detection method (review, unit test, system test, UAT, production), severity, priority. | Injected phase against detected phase gives the removal efficiency per phase (Task 3). Class shows what kind of defects dominate. |
| Impact | Module and feature, number of patients affected, safety / privacy / financial flag. | Separates a cosmetic issue from a wrong prescription. Supports severity decisions and incident reporting. |

| Group | Fields | Why they are needed |
|---|---|---|
| Cause and prevention | Root cause category (misunderstood rule, omission, boundary, concurrency, external change, test error), root cause note, preventive action agreed, checklist item added (yes/no). | Turns each defect into a lesson. A rule such as "check boundaries" enters the review checklist so the same error is not repeated. |
| Fix and verification | Fix description, change/commit ID, fixed-in build, effort to fix and to retest (hours), retest result, regression result. | Supplies real cost data to replace the assumed ₹500 to ₹40,000 figures and confirms the fix does not break something else. |
| Traceability | Linked requirement, linked test case, related or duplicate defects, link to external vendor ticket. | Shows which requirement was violated and which test should have found it, and groups related incidents (I1 and I6). |

### 9.2 How the repository supports analysis, improvement and prevention

- **Defect analysis:** Pareto charts by class, module and origin, and defects per release, show where defects cluster. Injected versus detected phase gives removal efficiency, which the team could not calculate before.
- **Process improvement:** a high count of requirements defects found late justifies a requirements review; a high count of escaped boundary defects leads to a boundary-testing standard; the measured fix hours replace assumptions in cost estimates.
- **Prevention:** root-cause and preventive-action fields feed checklists for reviews, coding standards and test design, so each incident (I3 for example) leaves a permanent barrier.
- **Release control:** the count of open severity 1 and 2 defects becomes an exit criterion instead of the "demo with no errors".

### 9.3 Weaknesses of the e-mail and spreadsheet approach

- No unique ID, no enforced workflow or history, so complaints are lost, duplicated or inconsistently closed.
- Free text with no mandatory fields: severity, origin and steps to reproduce are missing or vary by author, so aggregated analysis is impossible.
- No link to requirements, tests or code, so the team cannot see which test should have caught a defect.
- Reports are manual; management could not see the 60% escape rate or the 91.9% cost share.
- **Privacy risk:** patient names and details in e-mails and spreadsheets circulate without access control, which is unacceptable for a hospital.
- Version conflicts between several copies of the spreadsheet.

### 9.4 Appraisal of the benefits and costs

The repository costs the team a few extra minutes per defect and a monthly review meeting; a lightweight tracker (for example an issue tracker with custom fields) is sufficient for five developers. The benefit is the ability to measure, which every earlier task depended on. To protect patients, the repository must hold test or masked data, not real patient identities, and have role-

based access. The main limitation is data discipline: if fields are left empty, the analysis fails, so the origin, detection phase and root cause should be mandatory before a defect can be closed.

## TASK 10 – Evaluate Defect Examples and Prevention Strategies

Three incidents from three different classes are analysed: **I3 (requirements), I1 (design) and I4 (coding)**. Costs are estimates that use an assumed blended rate of ₹600 per person-hour.

| Defect | Root cause (why it happened) | Single process change that would have prevented it | Cost and effort against benefit |
|---|---|---|---|
| I3 – fee mismatch (Requirements) | The tax rule was taken from conversation, not from a written, approved rule. No finance review. The price was calculated in two places. | **Formal requirements inspection** with finance and OPD administration, using a checklist that demands a worked example for every calculation rule. | About 10 person-hours per session (₹6,000). A defect caught here costs ₹500 instead of ₹40,000: saving ₹39,500 per defect. Break-even is 0.15 defects, and I3 hit every payment. |
| I1 – duplicate tokens (Design) | Token allocation had no concurrency control (read-then-write); concurrent use was neither a requirement nor reviewed. | **Design inspection** with a concurrency and shared-resource checklist, leading to an atomic sequence or database unique constraint. | About 8 person-hours per session (₹4,800) plus about 1 to 2 days to implement. A design-stage fix costs ₹1,500 instead of ₹40,000: saving ₹38,500. The same checklist item also covers I6. |
| I4 – crash on long name (Coding) | No standard for input validation, no boundary tests; field lengths in screen, code and database were not aligned. | **Coding standard with mandatory input validation, enforced by peer code review and a boundary-value unit test**. | About 8 hours to write the standard and tests (₹4,800), then about 15 minutes per review. A coding-stage fix costs ₹3,000 instead of ₹40,000: saving ₹37,000. |

**Which strategy gives the best value?** All three pay back in less than one defect (break-even on review or set-up effort alone at about 0.15, 0.12 and 0.13 defects respectively), so none is a poor investment. Ranked by value, **requirements inspection is best**: it addresses the phase that produced 30% of the defects while detecting only 6.7%, it has the highest saving per defect (₹39,500), and it also removes the cause of problems in later phases, such as the unclear rules behind I5, I8 and I10. **Design inspection** is a close second because one checklist covers the two most dangerous incidents (I1 and I6) and also I9. The coding standard is third; it is cheap, but code reviews catch only a share of defects (the coding phase found 17.6% in the data). Test-oracle review (I8) is cheapest of all and should be done as well.

**Reconciling with Task 3:** a project of eight months would hold about five requirement and five design sessions, roughly ₹54,000, rounded up to the ₹60,000 assumed in Task 3 for follow-up effort.

## TASK 11 – Recommend and Defend a Quality Improvement Plan

**Principle of the plan:** contain the harm first, then build cheap early-phase defences, then add independent testing, then measure and improve. The order follows the evidence: 52% of defects

are born in requirements and design, early removal costs ₹500 to ₹1,500, and production costs ₹40,000.

### 11.1 Process changes in order of priority

| Step and timing | Actions | Evidence it rests on |
|---|---|---|
| 0. Contain (weeks 1–2) | Hotfix I6, I3, I1, I2 and I4; renew the SMS key and add an alert (I9); correct the I8 test case and audit all expected values; stop real patient data in complaint e-mails. | I6 and I3 are S1; production defects cost ₹40,000 each. |
| 1. Repository and glossary (weeks 1–3) | Introduce the Task 9 repository with mandatory origin, detection phase and severity; adopt the Task 4 glossary. | Without them nothing can be measured (Tasks 3, 4, 9). |
| 2. Requirements review (month 1 onward) | Written business-rule catalogue (opening days, fee and tax, senior-citizen rule, refund), reviewed by finance and OPD administration with a checklist and signed off. | Origin analysis (Task 7), cost ratio 1 : 80. |
| 3. Independent tester (month 1–2) | Appoint at least one tester reporting outside the developers; risk-based test plan; techniques (boundary values, equivalence classes, decision tables, concurrency tests); test cases traced to requirements with reviewed oracles. | Task 6: five high-likelihood and three medium-likelihood catches among the ten incidents. |
| 4. Design and code controls (month 2–3) | Design inspection with concurrency, external-interface and performance checklists; coding standard; peer review; unit tests on payment, token and prescription modules; static analysis in the build. | I1, I4, I5, I6, I9, I10 (Task 10). |
| 5. Test bed and release criteria (month 3–4) | Staging environment with SMS and payment sandboxes and a Wi-Fi-like network; performance test; written exit criteria (no open S1 or S2, all high-risk tests passed); manual tested against the build. | I7, I9, I10; the demo-only release rule (Task 1). |
| 6. Measure and improve (month 4–6) | Monthly defect review, removal efficiency per phase, escaped defects and cost; root-cause actions added to checklists. Target TMM Level 2 and then Level 3. | Task 2 maturity placement. |

### 11.2 Roles of testers, reviews and the repository

Reviews prevent and detect early defects at the cheapest stage. Testers provide independent test design, oracles and release evidence. The repository is the memory that connects both: it records where a defect was injected and detected, so the team can see whether reviews and tests are working and change them.

### 11.3 Expected effect on number and cost of defects

Using the same model as Task 3 and adding better coding controls (coding-phase removal rising from 17.6% to 40%) and an independent tester (testing-phase removal rising from 26.7% to 50%) together with the 60% and 50% review rates: defects detected before release would rise from 40 to 76, those reaching patients would fall from 60 to 24 (a 60% reduction), and the total fix cost would fall from ₹26,10,500 to about ₹12,12,500, a saving of ₹13,98,000 (53.6%). Review-only changes

alone give ₹7,62,000 (29.2%). These are estimates that rest on the assumed removal rates; they are not guarantees.

### 11.4 Trade-offs, risks and limitations

- Quality activities take time away from feature work. Expect roughly 15 to 25% of capacity in the first months. The ₹26,10,500 spent on defects in the last release suggests that this is repaid.
- A single tester can become a bottleneck or a "quality gate" that developers rely on. Developers keep responsibility for unit tests and reviews.
- Review fatigue and false confidence: reviews need preparation, short sessions and checklists that are updated from the repository.
- The estimates rest on one release and assumed rates; they must be revised with real data after two or three releases.
- Cost of a tester, tools and test-bed set-up is real but small against ₹24 lakh of production defect cost.

### 11.5 Is the plan practical for five developers?

Yes, because it is scaled to the team. Reviews are short (about one to two hours with a checklist) and use existing people, the tracker is a lightweight tool with a few mandatory fields, developers rotate as reviewers, and the only new head count is one tester (a ratio of one tester to five developers). The plan avoids heavy standards and starts with the three cheapest, highest-value steps: contain, requirements review and the repository.

## TASK 12 – Re-evaluate and Adapt the Quality Process

**Decision in brief: MODIFY.** Keep the foundations (early reviews, independent testing, the repository, classification and severity), but make them lighter, continuous and automated for a two-week cycle, and extend them for an external dependency and shared health records.

### A. Impact Analysis

**What has changed.**

- **Release rhythm:** from two releases a year to about 26. That is thirteen times more releases, so every release-time activity is repeated 13 times as often.
- **External dependency:** the national health ID system belongs to an agency with its own release schedule, versions, outages and rules. The team can no longer control or fully test the whole system.
- **Shared records:** patient health records are exchanged between hospitals, so data has legal, privacy and clinical consequences outside MediQueue.

**Assumptions of the Task 11 plan that no longer hold.**

- A long test phase of several weeks before release; the cycle now leaves only a few days.
- Whole-document reviews of requirements and design, which suit a six-month project but not weekly changes.

- Manual regression testing: repeating it 26 times a year is impractical.
- The team controls and can test all components in its own test bed.
- The repository records only internal defects, and the severity scale treats a privacy breach as one defect among others.
- Defect cost per phase (₹500 to ₹40,000): costs of defects involving shared records are likely to be higher than the table shows.

**Why frequent releases change the role of testing and of the tester.** Testing can no longer be a phase at the end; it becomes continuous, built into each sprint and run automatically on every build. The tester joins the sprint at story refinement to define acceptance criteria and risks, builds and maintains the automated regression suite with developers, does exploratory testing on new features, and supplies release evidence. The tester becomes a quality coach and risk analyst, not the person who runs manual cases at the end. Regression becomes the central risk: with a release every two weeks, each change can break what worked last time.

**Why sharing records raises severity.** Privacy and integrity failures now spread beyond the hospital: a disclosure like I6 would expose records across institutions and cannot be undone. A wrong patient match could attach another person's allergies or medication history to a patient and cause clinical harm in a different hospital. Data-protection law and consent obligations apply to the exchanged data. Therefore **data isolation, authorisation, patient identity matching, consent and concurrency defects move to S1 and block release**, and interface defects with the health ID service rise from S2 to S1 where they can mismatch identities or block care.

**B. Evaluation of the Original Plan**

| Element of the original plan | Fits a 2-week cycle? | Assessment |
|---|---|---|
| Formal requirements inspection of whole documents | Partly | Too slow as a gate for every change. Keep for high-risk rules (consent, identity matching, payment); use 20 to 30 minute story-level reviews with a checklist for the rest. |
| Design inspection | Partly | Do it once for the integration architecture and threat model, then per change with a short checklist (concurrency, external failure, privacy). |
| Independent tester executing system tests after development | No | No time in a two-week cycle; the tester must be in the sprint and rely on automation. |
| Manual regression | No | At even 5 person-days per release, 26 releases need 130 person-days a year, more than half of one person's working year. Must be automated. |
| Defect repository | Not sufficient | Has no field for external origin, API/contract version, vendor ticket, privacy/data-exposure flag, release or deployment ID, detection in production, time to detect and time to restore. |
| Per-release removal-efficiency metrics | Weak | A two-week release has too few defects for stable percentages; use rolling averages across releases and track escaped defects per release. |

**New defect origins introduced by the external health ID system:**

- **Interface and contract defects:** field formats, mandatory fields, API versions and deprecations differ from what MediQueue expects.
- **Behavioural differences between sandbox and production,** and rules that are ambiguous or undocumented.
- **Availability, latency and rate-limit problems** of a service the team cannot fix, and the effect on booking and registration.
- **Data quality in the national registry:** duplicate or missing IDs, name or date-of-birth formats that do not match local records (identity matching errors).
- **Security and credentials:** token expiry, certificate and key rotation (the larger version of I9).
- **Unannounced or late-announced changes** by the agency.
- **Consent and data-sharing scope errors:** sharing more than the patient agreed or less than care requires.

**C. Adaptation or Redesign**

**Decision: MODIFY.** The core ideas stay because the defect profile has not changed: most defects still originate upstream, independent verification and an oracle are still needed, and measurement still depends on the repository. What changes is the speed, the automation and the treatment of the external boundary.

**Why not retain?** Because three assumptions fail (long test phase, manual regression, full control of the test bed) and the repository cannot describe external defects. **Why not redesign completely?** Because the evidence of Tasks 1 to 11 still supports early reviews and independence, and scrapping them would waste scarce capacity of five developers.

**Revised process (two-week sprint).**

- **Day 1 planning:** story-level requirement review, risk analysis (which stories touch identity, consent, payment, token, prescription), acceptance criteria and test ideas defined before coding (the "definition of ready").
- **Days 2 to 8:** coding with peer review, unit tests and static analysis on every build; contract tests against the health ID specification and sandbox; automated checks for concurrency and data isolation between hospitals.
- **Days 8 to 9 hardening:** automated regression of critical flows (book, pay, token, prescription, cancel, refund, consent), performance smoke test on a Wi-Fi-like network, exploratory testing and a privacy and authorisation test.
- **Day 10 release and monitoring:** staged release with feature flags, monitoring and alerts on SMS and health ID calls, a rollback plan. Exit criteria: no open S1 or S2, critical flows and contract tests pass; otherwise the feature stays off.
- **Retrospective:** review the repository: escaped defects, root causes, new checklist items.

**Tester's revised role.** Embedded in the sprint (about one tester for five developers, with someone responsible for privacy and security testing); defines acceptance criteria and risk; builds automation

and contract tests with developers; performs exploratory and privacy testing; owns release evidence and the repository analysis; and is the contact with the agency for interface problems. Developers own unit and component tests.

**Testing principles that become more important.**

- **Early testing:** only possible within two weeks if questions are asked at story refinement.
- **Regression and the pesticide paradox:** the automated suite must be updated so it does not stop finding new defects.
- **Risk-based selection (exhaustive testing impossible):** with limited days, tests focus on identity, consent, payment, token and concurrency.
- **Context dependence:** shared health records make privacy and clinical safety the dominant context.
- **Absence-of-errors fallacy:** compliance with the agency specification and real clinical workflow must be validated, not just matched to the spec.

**Classification and tracking of external defects.** Add classes: external interface/contract, external availability and performance, external data quality, external change and security/consent. Add repository fields: origin = external, system and API version, owner (agency or hospital), vendor ticket number and SLA, workaround, and a privacy flag; mask all patient identifiers. Triage rule: first decide whether the fault is ours (misuse of the interface), theirs, or ambiguous specification; whatever the answer, **log an internal defect for MediQueue's handling of the failure** (the lesson of I9: a failing external service must not fail silently). Severity is based on patient impact regardless of origin. Track monthly counts by external origin, time to detect and time to resolve, and keep a log of agency changes.

**D. Evidence-Based Defence**

- **Defect data and cost.** 91.9% of the previous cost came from 60 production defects at ₹40,000 each; the cost ratio is 1 : 80 between requirements and production, and it can only grow when a production defect involves shared patient data. This supports keeping the early checks and adding automated gates, not removing them.
- **Origins and classes.** 52% of defects were introduced in requirements and design and 9 of 10 incidents had an upstream contribution. Hence story-level reviews stay, although lighter. The new origin classes (contract, identity, consent) are extensions of the same pattern: an unclear interface rule is a requirements defect.
- **Release frequency.** Twenty-six releases a year make manual regression unaffordable: 5 person-days per release would be 130 person-days a year. If building a suite of critical-flow tests takes about 35 person-days and regression effort falls to about 1 day per release, the saving of about 4 days a release pays back in about 9 releases (around 4 to 5 months). These figures are illustrative assumptions.
- **Patient safety and privacy.** I6 shows that data leaks already occurred in a single hospital. With shared records the consequence is multiplied, so S1 gating, authorisation and isolation tests, staged release and monitoring are justified even at a cost to speed.

- **Team size and capacity.** Five developers give about 50 developer-days per sprint. Quality activities (story reviews, unit tests, peer review) of about 15 to 20% plus one embedded tester are feasible; heavy phase-based documents are not, which is why they are lightened.

**Why this decision follows from the evidence.** The evidence of the last release (early defects, late detection, no independent oracle, no measurement) is not changed by faster releases; it is made more dangerous by them. The retained controls address the causes. The modifications address only what the new conditions break: time (automation, story-level reviews), control (contract tests, monitoring, external classes) and consequence (higher severity for privacy and identity). A redesign would discard validated controls; retaining the plan unchanged would fail on time and the external boundary.

**Risks and how they will be checked.** The up-front automation effort will slow the first two or three sprints; the sandbox of the external system may differ from production; the agency can change an interface without notice; the removal-efficiency data are limited by small samples. After six releases the decision should be checked against targets such as zero S1 defects escaping, pre-release removal efficiency rising from 40% to at least 70%, at least 80% of critical flows automated, and a stable or falling count of escaped defects per release. If these are not met, the plan is adjusted, not abandoned.